γραφείου βασίζονται σε σταθερό μισθό 1200 Ευρώ τον μήνα. Επιπλέον, κάθε υπάλληλος γραφείου λαμβάνει μηνιαία ένα ποσό που υπολογίζεται ως το γινόμενο 5% x Αριθμός ετών στην εταιρεία x σταθερός μισθός.

Τα διευθυντικά στελέχη αμείβονται με σταθερό μισθό 3500 Ευρώ συν ένα ποσό που λαμβάνουν επίσης μηνιαία και έχει συμφωνηθεί μεταξύ εταιρείας και κάθε στελέχους ξεχωριστά.

Οι συμβασιούχοι αμείβονται με ένα ποσό ανά ώρα εργασίας που συμφωνείται μεταξύ της εταιρείας και κάθε συμβασιούχου ξεχωριστά. Κατά την έκδοση της μισθοδοσίας μηνός, οι συμβασιούχοι ενημερώνουν για τον αριθμό ωρών που εργάστηκαν κατά τη διάρκεια του μήνα.

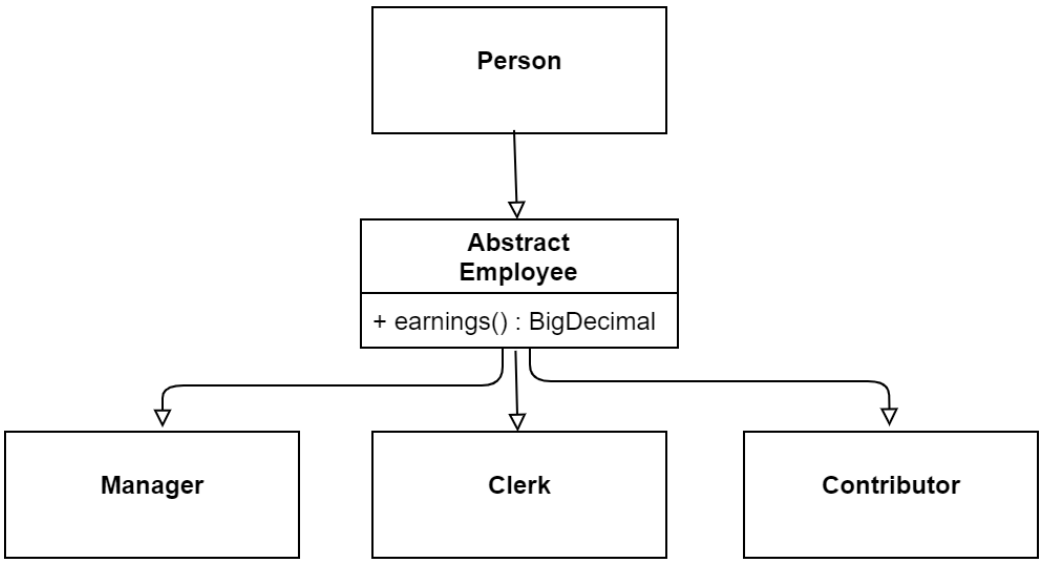
Η εταιρεία διαθέτει τρεις υπαλλήλους γραφείου, δύο διευθυντές και δύο συμβασιούχους. Οι διευθυντές έχουν συμφωνήσει ως επιπρόσθετο μηνιαίο ποσό 2000 Ευρώ ο ένας και 2200 ο άλλος. Οι συμβασιούχοι έχουν συμφωνήσει ωρομίσθιο 12 Ευρώ ο πρώτος και 20 ο δεύτερος. Ο πρώτος εργάστηκε 120 ώρες και ο δεύτερος 140 ώρες αυτόν τον μήνα. Τέλος, οι υπάλληλοι έχουν προσληφθεί σε τυχαίες ημερομηνίες μέσα στις 10.000 προηγούμενες ημερομηνίες από την τρέχουσα.

Η εταιρεία θέλει να υλοποιήσετε εφαρμογή που υπολογίζει και εμφανίζει κατάσταση μισθοδοσίας του τρέχοντος μήνα. Η κατάσταση σε κάθε γραμμή θα πρέπει να περιλαμβάνει τον αριθμό ταυτότητας του υπαλλήλου, το ονοματεπώνυμό του και τις αποδοχές του μήνα.

Λύση

Όπως διατυπώνεται στην εκφώνηση, οι εργαζόμενοι στην εταιρεία είναι φυσικά πρόσωπα. Ωστόσο, ως εργαζόμενοι στην εταιρεία πρέπει να είναι σε θέση να υπολογίζουν το μηνιαίο εισόδημά τους. Επομένως, επιτελούν μια επιπλέον λειτουργία, δηλαδή μια λειτουργία που δεν συναντάται σε πρόσωπα που δεν είναι εργαζόμενοι. Από την άλλη πλευρά, ο υπολογισμός του μηνιαίου εισοδήματος βασίζεται σε εντελώς διαφορετικά δεδομένα και είναι γενικότερα διαφορετικός για κάθε τύπο εργαζόμενου.

Με αυτά τα δεδομένα, μας εξυπηρετεί η οργάνωση των κλάσεων, όπως φαίνεται στο σχήμα 13.3.



Σχήμα 13.3 *Διάγραμμα κλάσεων της εφαρμογής Μισθοδοσία Μηνός*

Ο εργαζόμενος αναπαρίσταται από την κλάση Employee, παράγωγη κλάση της Person. Ωστόσο, πρόκειται για αφηρημένη κλάση, καθώς δηλώνει τη συνάρτηση earnings που επιστρέφει το μηνιαίο εισόδημα του εργαζόμενου. Η συνάρτηση αυτή όμως δεν μπορεί να οριστεί σαφώς στο επίπεδο του εργαζόμενου. Μπορεί όμως να οριστεί στις παράγωγες κλάσεις της Employee που αναπαριστούν εργαζόμενους συγκεκριμένου τύπου. Η κλάση Manager αναπαριστά τους διευθυντές, η κλάση Clerk τους υπαλλήλους γραφείου, η κλάση Contributor τους συμβασιούχους.

Ας δούμε καταρχάς την ανάπτυξη των κλάσεων και στη συνέχεια την εφαρμογή μισθοδοσία μηνός.

```
import java.math.BigDecimal;
import java.time.LocalDate;

public abstract class Employee extends Person {

    public Employee(String id, String fName, String sName, LocalDate birthday) {
        super(id, fName, sName, birthday);
    }

    public abstract BigDecimal earnings();
}
```

Κώδικας 13.25 *Η αφηρημένη κλάση Employee*

Η κλάση Employee έχει ένα ιδιαίτερο χαρακτηριστικό. Κληρονομεί την κλάση Person και ταυτόχρονα είναι αφηρημένη. Παρότι δεν είναι εφικτή η δημιουργία αντικειμένων της κλάσης, απαιτείται να οριστεί δημιουργός ο οποίος εκείνο που κάνει είναι ότι καλεί τον δημιουργό της γονικής Person. Ο δημιουργός εδώ είναι απαραίτητος ώστε οι παράγωγες κλάσεις Manager, Clerk και Contributor να μπορούν να τον καλέσουν και αυτός με τη σειρά του να καλέσει τον δημιουργό Person. Κατά τα λοιπά, η κλάση δηλώνει την αφηρημένη συνάρτηση earnings που υπολογίζει το μηνιαίο εισόδημα του εργαζόμενου.

```
import java.math.BigDecimal;
import java.time.LocalDate;

public class Manager extends Employee {
    private final BigDecimal constantFee=new BigDecimal(3500);
    private final BigDecimal bonus;

    public Manager(BigDecimal bonus, String id, String fName, String sName,
LocalDate birthday) {
        super(id, fName, sName, birthday);
        this.bonus = bonus;
    }

    @Override
    public BigDecimal earnings() {
        return constantFee.add(bonus);
    }
}
```

Κώδικας 13.26 *Η κλάση Manager*

Ο δημιουργός της κλάσης Manager καλεί τον δημιουργό της Employee και στη συνέχεια δίνει τιμή στην πρόσθετη αμοιβή του στελέχους. Σύμφωνα με την εκφώνηση, η πρόσθετη αμοιβή καθορίζεται σε συμφωνία με την εταιρεία ξεχωριστά για κάθε στέλεχος. Επομένως, μπορεί η τιμή αυτή να καθοριστεί κατά τη δημιουργία των αντικειμένων Manager, ειδικά για κάθε αντικείμενο.

Στη συνέχεια, υλοποιείται η συνάρτηση earnings που απλώς επιστρέφει το άθροισμα του μηνιαίου μισθού συν την πρόσθετη αμοιβή.

Η κλάση Clerk παρουσιάζεται στον κώδικα 13.27:

```
import java.math.BigDecimal;
import java.time.LocalDate;
import java.time.Period;
import java.util.Random;

public class Clerk extends Employee {
    private static final BigDecimal BASESALARY=new BigDecimal(1200);
    private static final double INCREMENTRATE=0.05;
```

```
    private final LocalDate hireDate;

    public Clerk(String id, String fName, String sName, LocalDate birthday)
{
        super(id, fName, sName, birthday);
        hireDate=LocalDate.now().minusDays(new Random().nextInt(10000));
    }


    @Override
    public BigDecimal earnings() {
        LocalDate today = LocalDate.now();
        int workingYears=Period.between(hireDate, today).getYears();
        BigDecimal incrementAmount=BASESALARY.multiply(new
BigDecimal(workingYears*INCREMENTRATE));
        double d=incrementAmount.doubleValue();
        BigDecimal salary=BASESALARY.add(incrementAmount);
        return salary;
    }
}
```

Κώδικας 13.27 *Η κλάση Clerk*

Οι υπάλληλοι γραφείου αμείβονται με σταθερό μηνιαίο μισθό συν μια προσαύξηση που εξαρτάται από ένα σταθερό ποσοστό και από τα έτη προϋπηρεσίας στην εταιρεία. Εφόσον ο μηνιαίος μισθός και το ποσοστό προσαύξησης είναι σταθερά ποσά, οι μεταβλητές BASESALARY και INCREMENTRATE δηλώθηκαν ως final. Επιπλέον, τόσο ο μηνιαίος μισθός όσο και το ποσοστό προσαύξησης είναι κοινά για όλους τους υπαλλήλους γραφείου, επομένως, οι αντίστοιχες μεταβλητές θα πρέπει να δηλωθούν ως static, καθώς αφορούν την κλάση και όχι κάθε αντικείμενό της χωριστά. Η ημερομηνία πρόσληψης όμως αφορά κάθε εργαζόμενο χωριστά και δηλώνεται ως μεταβλητή στιγμιότυπου. Καθώς και αυτή δεν είναι λογικό να αλλάξει μετά την πρόσληψη εργαζόμενου, μπορεί επίσης να δηλωθεί ως final.

Ο δημιουργός της κλάσης, αφού καλέσει τον δημιουργό της γονικής, δίνει τιμή στην ημερομηνία πρόσληψης σύμφωνα με τις απαιτήσεις της εκφώνησης.

Στη συνέχεια, η κλάση επανορίζει τη συνάρτηση earnings. Υπολογίζει τα έτη από την ημερομηνία πρόσληψης μέχρι σήμερα, μετά, το ποσό προσαύξησης, το οποίο προσθέτει στον βασικό μισθό και επιστρέφει τις αποδοχές του μήνα.

Τέλος, οι συμβασιούχοι αναπαρίστανται από την κλάση Contributor (κώδικας 13.28):

```
import java.math.BigDecimal;
import java.time.LocalDate;

public class Contributor extends Employee {

    private int workingHours;
    private BigDecimal paymentPerHour;

    public Contributor(BigDecimal paymentPerHour, String id, String fName,
String sName, LocalDate birthday) {
        super(id, fName, sName, birthday);
        this.paymentPerHour = paymentPerHour;
    }

    public void setWorkingHours(int workingHours) {
        this.workingHours = workingHours;
    }

    @Override
    public BigDecimal earnings() {
        return paymentPerHour.multiply(new BigDecimal(workingHours));
    }
```

```
}
```

Κώδικας 13.28 *Η κλάση Contributor*

Οι συμβασιούχοι αμείβονται με βάση μια αμοιβή για την ώρα εργασίας τους που συμφωνείται μεταξύ εταιρείας και κάθε συμβασιούχου χωριστά. Η πληροφορία συνοδεύει κάθε συμβασιούχο από τη δημιουργία του και επομένως μπορεί να δοθεί στον δημιουργό. Οι ώρες εργασίας όμως αφορούν τις ώρες εργασίας του μήνα, πιθανότατα παρουσιάζουν διακυμάνσεις από μήνα σε μήνα και άρα δεν σχετίζονται με τη δημιουργία του αντικειμένου. Έτσι παρέχουμε τον ρυθμιστή setWorkingHours ώστε να ενημερώνεται κατάλληλα η αντίστοιχη μεταβλητή. Τέλος, η earnings επιστρέφει το γινόμενο ώρες εργασίας επί αμοιβή ανά ώρα.

Έχοντας διαθέσιμες τις κλάσεις, η ανάπτυξη της εφαρμογής είναι σχετικά εύκολη.

```
import java.math.BigDecimal;
import java.text.DecimalFormat;
import java.text.DecimalFormatSymbols;
import java.time.LocalDate;
import java.time.Month;
import java.util.ArrayList;

public class CalcMonthlyPayment { //Κώδικας 13.29

    public static void main(String[] args) {
        ArrayList<Employee> personel = new ArrayList<>();
        personel.add(new Clerk("1", "Jim", "Black", LocalDate.of(1967,
Month.MARCH, 1)));
        personel.add(new Clerk("2", "John", "Blue", LocalDate.of(1972,
Month.APRIL, 12)));
        personel.add(new Clerk("3", "Jack", "Dalton", LocalDate.of(1980,
Month.MARCH, 21)));
        personel.add(new Manager(BigDecimal.valueOf(2000), "101", "Adam",
"Smith", LocalDate.of(1980, Month.JANUARY, 12)));
        personel.add(new Manager(BigDecimal.valueOf(2200), "102", "Adam",
"Smith", LocalDate.of(1980, Month.MARCH, 21)));
        Contributor cntr = new Contributor(BigDecimal.valueOf(12), "1001",
"Kalamity", "Jane", LocalDate.of(2000, Month.MARCH, 21));
        cntr.setWorkingHours(120);
        personel.add(cntr);
        cntr = new Contributor(BigDecimal.valueOf(20), "1002", "Erick",
"Erickson", LocalDate.of(2001, Month.MARCH, 21));
        cntr.setWorkingHours(140);
        personel.add(cntr);

        DecimalFormatSymbols dfs = new DecimalFormatSymbols();
        dfs.setDecimalSeparator('.');
        dfs.setGroupingSeparator(',');
        DecimalFormat f = new DecimalFormat("######.###");
        f.setDecimalFormatSymbols(dfs);

        for (Employee e : personel) {
            System.out.println(e.getId() + " " + e.getfName() + " " +
e.getsName() + " " + f.format(e.earnings().doubleValue()));
        }
    }
}
```

Κώδικας 13.29 *Η εφαρμογή CalcMonthlyPayment*

Καταρχάς, η εφαρμογή δημιουργεί ένα ArrayList για Employee. Στη συνέχεια δημιουργεί ένα αντικείμενο για κάθε εργαζόμενο και το προσθέτει στο ArrayList. Ειδικά για τα αντικείμενα Contributor πριν τα προσθέσει στο ArrayList, ενημερώνει τις ώρες εργασίας. Στη συνέχεια, φροντίζει για την κατάλληλη διαμόρφωση της εξόδου και μετά τυπώνει την κατάσταση μισθοδοσίας βασιζόμενη απλώς στις συναρτήσεις των κλάσεων. Οι getId,

getfName και getsName έχουν κληρονομηθεί αυτούσιες από την κλάση Person. Η earnings, όπως είδαμε, ορίζεται στις σαφείς κλάσεις εργαζόμενων. Προκειμένου να αξιοποιήσουμε την format, μετατρέπουμε τα earnings από BigDecimal σε double.

13.5 Ασκήσεις προς Λύση

13.5.1 Ο ρόμβος

Στην άσκηση 13.4.1 υπολογισμού εμβαδού γεωμετρικών σχημάτων να υποστηριχτεί ο υπολογισμός εμβαδού του ρόμβου. Ο υπολογισμός του εμβαδού ρόμβου μπορεί να γίνει συνάρτηση των μηκών των διαγώνιων του σύμφωνα με τον τύπο που ακολουθεί

$$E = \frac{(\delta 1 \times \delta 2)}{2}, \delta 1 \; και \; \delta 2 \; οι \; διαγώνιοι$$

Επιπλέον, προσθέστε στις κλάσεις της άσκησης όλες τις αναγκαίες μεθόδους που έχουν παραληφθεί, π.χ. equals, hashCode, toString, ρυθμιστές και αναγνώστες.

13.5.2 Ζωολογικός κήπος

Σε έναν ζωολογικό κήπο έχουν συγκεντρωθεί διάφορα ζώα. Από αυτά, κάποια περπατούν, κάποια πετούν, κάποια κολυμπούν και κάποια έρπουν. Κατασκευάστε έναν ζωολογικό κήπο, προσθέστε ζώα από κάθε τύπο και θέστε τα σε κίνηση. Οι συναρτήσεις κίνησης θα βγάζουν μηνύματα που απλώς περιγράφουν την κίνηση, π.χ. το λιοντάρι περπατά, το δελφίνι κολυμπά.

13.5.3 Η συμπεριφορά των μεταβλητών αναφοράς

Στην άσκηση 13.4.3 ενημερώνουμε πρώτα τις ώρες εργασίας ενός αντικειμένου τύπου Contributor και στη συνέχεια προσθέτουμε το αντικείμενο στο ArrayList. Τι θα συμβεί αν κάνουμε το αντίθετο, δηλαδή αν προσθέσουμε στο ArrayList το αντικείμενο και στη συνέχεια ενημερώσουμε τις ώρες εργασίας; Ελέγξτε και εξηγήστε τη συμπεριφορά.

13.5.4 Βαθμολογία

Σε ένα τμήμα της Γ' Γυμνασίου φοιτούν 11 μαθητές. Από αυτούς, οι τρεις είναι αλλοδαποί και οι υπόλοιποι ημεδαποί. Όλοι οι μαθητές παρακολουθούν τα μαθήματα Άλγεβρας, Φυσικής και Γλώσσας. Οι αλλοδαποί παρακολουθούν επιπλέον το μάθημα Εκμάθηση της Ελληνικής γλώσσας. Όλοι οι μαθητές μπορούν να παρακολουθήσουν ή όχι Θρησκευτικά. Το μάθημα έχει επιλέξει ένας αλλοδαπός και έξι ημεδαποί. Όλοι οι μαθητές είναι φυσικά πρόσωπα. Βρισκόμαστε στο τέλος της περιόδου των μαθημάτων, οι μαθητές έχουν βαθμολογηθεί και πρέπει να υπολογίσουμε τον μέσο όρο βαθμολογίας κάθε μαθητή. Φορτώστε σε ένα ArrayList τους 11 μαθητές και εμφανίστε για κάθε έναν τον αριθμό ταυτότητας, το ονοματεπώνυμό του, την αναλυτική βαθμολογία του και τον μέσο όρο του.

Βιβλιογραφία

[1] M. Fowler, UML Distilled: A Brief Guide to the Standard Object Modeling Language, 3rd edition. Boston: Addison-Wesley Professional, 2003.

Κεφάλαιο 14

Σύνοψη
Σε αυτήν την ενότητα καταρχάς εξηγείται η έννοια της διεπαφής στην Java. Στη συνέχεια παρουσιάζονται βασικές προκαθορισμένες διεπαφές και η χρήση τους. Αναλύεται η υλοποίηση διεπαφών οριζόμενων από τον χρήστη. Συζητούνται τα προβλήματα της πολλαπλής κληρονομικότητας και αναλύεται ο μηχανισμός μέσα από τον οποίον υποστηρίζει η Java την πολλαπλή κληρονομικότητα. Εξηγείται η κληρονομικότητα ανάμεσα σε διεπαφές, η ταυτόχρονη κληρονομικότητα από κλάση και διεπαφή, τα στατικά μέλη στις διεπαφές, οι προκαθορισμένες μέθοδοι και οι ανώνυμες κλάσεις.

Προαπαιτούμενη γνώση
Η ενσωμάτωση, η κληρονομικότητα και ο πολυμορφισμός στο αντικειμενοστρεφές μοντέλο με Java, οι αφηρημένες κλάσεις καθώς και οι βασικές προκαθορισμένες κλάσεις, δηλαδή οι String, StringBuilder, ArrayList, LinkedList, Stack, PriorityQueue, HashSet, HashMap, HashTree, LocalDate, Localtime, Period και Duration.

Λέξεις κλειδιά
Διεπαφή (interface), πρόβλημα του διαμαντιού (the diamond problem), πολλαπλή κληρονομικότητα (multiple inheritance), κληρονομικότητα κατάστασης (inheritance of state), κληρονομικότητα υλοποίησης (inheritance of inheritance), κληρονομικότητα τύπου (state inheritance), προκαθορισμένες μέθοδοι (default methods), ανώνυμες κλάσεις (anonymous class).

14 Διεπαφές

Σε πολλές περιπτώσεις, μια ομάδα μηχανικών λογισμικού χρειάζεται να χρησιμοποιήσει λογισμικό που έχει αναπτύξει άλλη ομάδα. Στις περιπτώσεις αυτές χρειάζεται ένας τυπικός τρόπος με τον οποίο η δεύτερη ομάδα πληροφορεί την πρώτη για τα χαρακτηριστικά του λογισμικού της. Αυτός ο τρόπος παρέχεται δια μέσου των διεπαφών (interfaces) [1]. Με άλλα λόγια, όταν ένα λογισμικό Β καλεί ένα λογισμικό Α, τότε το Β χρειάζεται να γνωρίσει τα χαρακτηριστικά του Α. Αυτά ακριβώς τα χαρακτηριστικά εκθέτει το Α προς τους τρίτους δια μέσου των διεπαφών. Πρόκειται δηλαδή για μια συμφωνία μεταξύ των δύο ομάδων μηχανικών λογισμικού ή ακριβέστερα μεταξύ δύο λογισμικών με την οποία καθορίζονται τα χαρακτηριστικά του λογισμικού ώστε η ομάδα-χρήστης να μπορεί να το χρησιμοποιεί χωρίς να γνωρίζει τις εσωτερικές λεπτομέρειές του. Με αυτήν την έννοια, οι διεπαφές αποτελούν έναν ακόμη μηχανισμό αφαιρετικότητας. Αρχικά, οι διεπαφές της Java συνιστούσαν ένα είδος απολύτως αφηρημένης κλάσης, δηλαδή μίας κλάσης στην οποία όλες οι μέθοδοι στιγμιότυπου ήταν αφηρημένες. Από την έκδοση 8 και μετά προστέθηκαν οι προκαθορισμένες μέθοδοι (default methods) που είναι σαφείς. Οι διεπαφές εκτός από αφηρημένες και προκαθορισμένες μεθόδους μπορεί να περιέχουν και στατικές μεθόδους, σταθερές και εμφωλευμένους τύπους. Δεν μπορούν όμως να περιέχουν μεταβλητές στιγμιότυπου. Οι διεπαφές είναι τύποι αναφοράς, ωστόσο δεν μπορεί να δημιουργηθούν στιγμιότυπα διεπαφής για τον ίδιο λόγο που δεν μπορεί να δημιουργηθούν στιγμιότυπα αφηρημένης κλάσης. Μπορούν όμως να δημιουργηθούν στιγμιότυπα των κλάσεων που υλοποιούν (implements) τις διεπαφές. Κάθε κλάση που υλοποιεί μια διεπαφή συνδέεται με τη διεπαφή με τη σχέση “είναι ένα”.

Οι διεπαφές μπορούν να κληρονομούν άλλες διεπαφές. Μάλιστα μια διεπαφή μπορεί να έχει πολλαπλές γονικές διεπαφές. Οι διεπαφές όμως δεν μπορούν να κληρονομήσουν μία κλάση.

Ένα άλλο χαρακτηριστικό των διεπαφών είναι πως για κάθε δημόσια μέθοδο της κλάσης Object, μία αφηρημένη μέθοδος με ίδια ταυτότητα δηλώνεται αυτόματα σε κάθε διεπαφή. Για παράδειγμα, αν δούμε την τεκμηρίωση μιας προκαθορισμένης διεπαφής List θα βρούμε τη μέθοδο boolean equals(Object o). Δημιουργείται έτσι λανθασμένα η εντύπωση πως οι διεπαφές κληρονομούν την κλάση Object. Γενικά οι διεπαφές δεν κληρονομούν από κλάσεις. Ωστόσο, μια διεπαφή μπορεί να κληρονομεί από μία ή περισσότερες διεπαφές.

Πέραν των ανωτέρω, οι διεπαφές αποτελούν τον μηχανισμό μέσα από τον οποίον η Java επιτυγχάνει την πολλαπλή κληρονομικότητα τύπου ενώ ενισχύουν τις δυνατότητες πολυμορφικού κώδικα.

Όλα αυτά θα τα δούμε στη συνέχεια αναλυτικά, αρχίζοντας με την παρουσίαση κάποιων βασικών διεπαφών της βιβλιοθήκης της Java.

14.1 Βασικές διεπαφές

Στις βιβλιοθήκες της Java πέραν των κλάσεων έχουν ορισθεί και πάρα πολλές διεπαφές. Παρουσιάζουμε εδώ συγκεκριμένα τη διεπαφή Iterator και Comparator. Αρχικά μας ενδιαφέρουν κυρίως οι αφηρημένες μέθοδοί τους. Παρακάτω σε αυτό το κεφάλαιο θα συζητήσουμε για τα υπόλοιπα χαρακτηριστικά των διεπαφών.

14.1.1 Iterator

Ο Iterator είναι ίσως η συχνότερα χρησιμοποιούμενη διεπαφή. Μας δίνει τη δυνατότητα να προσπελάσουμε ένα προς ένα τα στοιχεία μιας συλλογής. Δεδομένου ότι οι συλλογές αποτελούν γενικεύσεις, έτσι και ο Iterator αποτελεί μια γενίκευση, π.χ. για να προσπελάσουμε μέσω ενός Iterator τα στοιχεία ενός ArrayList<Person>, χρειαζόμαστε έναν Iterator<Person>. Γενικά, κάθε Iterator αφορά κάποια συλλογή.
Ο Iterator ορίζει δύο αφηρημένες μεθόδους:

```
boolean hasNext()
```

Ελέγχει αν υπάρχει επόμενο στοιχείο στη συλλογή

```
E next()
```

Επιστρέφει το επόμενο διαθέσιμο στοιχείο της συλλογής.

Κάθε μια από τις προκαθορισμένες συλλογές διαθέτει μια μέθοδο που επιστρέφει έναν Iterator με τον οποίο μπορούμε να προσπελάσουμε τα στοιχεία της. Ακολουθεί ένα παράδειγμα χρήσης Iterator με ArrayList.

```
import gr.ihu.cs.lmous.OOJ.k12.v1.x.Person;
import java.time.LocalDate;
import java.time.Month;
import java.util.ArrayList;
import java.util.Iterator;

public class IteratorDemo {

    public static ArrayList<Person> loadData() {
        ArrayList<Person> rVal = new ArrayList<>();
        rVal.add(new Person("10", "Jim", "Smith", LocalDate.of(1961,
Month.JANUARY, 1)));
        rVal.add(new Person("20", "Adam", "Smith", LocalDate.of(1965,
Month.JANUARY, 1)));
        rVal.add(new Person("1", "John", "Black", LocalDate.of(1971,
Month.JANUARY, 1)));
        rVal.add(new Person("2", "John", "Dalton", LocalDate.of(1965,
Month.JANUARY, 1)));
        rVal.add(new Person("12", "Bill", "Adams", LocalDate.of(1965,
Month.JANUARY, 5)));
        rVal.add(new Person("6", "Averell", "Dalton", LocalDate.of(1972,
Month.JANUARY, 1)));
        rVal.add(new Person("7", "John", "Cyan", LocalDate.of(1980,
Month.JANUARY, 1)));
        return rVal;
    }

    public static void main(String[] args) {
        ArrayList<Person> store = loadData();
        Iterator<Person> it = store.iterator();
        while (it.hasNext()) {
            System.out.println(it.next());
        }
    }
}
```

Κώδικας 14.1 *Παράδειγμα χρήσης Iterator με ArrayList.*

Στην main του κώδικα 14.1 καλείται καταρχάς η loadData που φορτώνει στο ArrayList, store, τα δεδομένα 7 προσώπων. Η κλάση ArrayList διαθέτει τη μέθοδο iterator που επιστρέφει έναν Iterator για πρόσωπα. Καλώντας την iterator λαμβάνουμε έναν τέτοιον Iterator και τον τοποθετούμε στην αναφορά it. Διαμέσου αυτής της αναφοράς προσπελαύνουμε ένα-ένα τα στοιχεία της ArrayList με την it.next(). Αυτή θα πρέπει να καλείται μόνον εφόσον έχουμε ελέγξει πως υπάρχει διαθέσιμο επόμενο στοιχείο, δηλαδή μόνον εφόσον η it.hasNext() επιστρέψει true. Να σημειωθεί πως σε αυτό το παράδειγμα, η it.next() επιστρέφει ένα αντικείμενο τύπου Person.

Ας δούμε και ένα παράδειγμα που θα βοηθήσει να κατανοήσουμε τις δυνατότητες πολυμορφικού κώδικα που παρέχονται μέσω των διεπαφών.

```
import java.time.LocalDate;
import java.time.Month;
import java.util.ArrayList;
import java.util.Iterator;
import java.util.LinkedList;
import java.util.List;

public class PolymorphismDemo {

      public static List<Person> loadData() {
        List<Person> rVal = new ArrayList<>();
        rVal.add(new Person("10", "Jim", "Smith", LocalDate.of(1961,
Month.JANUARY, 1)));
        rVal.add(new Person("20", "Adam", "Smith", LocalDate.of(1965,
Month.JANUARY, 1)));
        rVal.add(new Person("1", "John", "Black", LocalDate.of(1971,
Month.JANUARY, 1)));
        rVal.add(new Person("2", "John", "Dalton", LocalDate.of(1965,
Month.JANUARY, 1)));
        rVal.add(new Person("12", "Bill", "Adams", LocalDate.of(1965,
Month.JANUARY, 5)));
        rVal.add(new Person("6", "Averell", "Dalton", LocalDate.of(1972,
Month.JANUARY, 1)));
        rVal.add(new Person("7", "John", "Cyan", LocalDate.of(1980,
Month.JANUARY, 1)));
        return rVal;
    }

    public static void main(String[] args) {
        List<Person> store = loadData();
        for (int i=0; i<2; i++) {
            Iterator<Person> it = store.iterator();
            while (it.hasNext()) {
                System.out.println(it.next());
            }
            System.out.println("-------------------------------------");
            store = new LinkedList<>(store);
        }
    }
}
```

Κώδικας 14.2 *Προσπέλαση των στοιχείων μιας ArrayList και μιας LinkedList με τον ίδιο κώδικα*

Στον κώδικα 14.2 η loadData έχει τροποποιηθεί ώστε ο τύπος επιστροφής της δεν είναι ArrayList<Person> όπως στον κώδικα 14.1 αλλά List<Person>. Η List είναι προκαθορισμένη διεπαφή. Τόσο η κλάση ArrayList όσο και η κλάση LinkedList υλοποιούν τη διεπαφή List. Αυτό σημαίνει πως μια ArrayList είναι μια List και μια LinkedList είναι επίσης μία List. Επομένως, η loadData που η τυπική τιμή επιστροφής της είναι List<Person> μπορεί να επιστρέψει πραγματικό αντικείμενο τύπου ArrayList<Person> ή LinkedList<Person> ή οποιασδήποτε άλλης συλλογής που υλοποιεί την List.

Στην main ορίζουμε επίσης τη μεταβλητή store ως List<Person>. Επομένως στη μεταβλητή αυτή μπορεί να εκχωρηθεί αναφορά σε πραγματικό αντικείμενο οποιασδήποτε κλάσης υλοποιεί την List. Αν μεταβούμε στην τεκμηρίωση της List θα διαπιστώσουμε ότι ανάμεσα σε άλλες μεθόδους ορίζει και μία μέθοδο Iterator<E> iterator(). Κάθε σαφής κλάση που υλοποιεί τη διεπαφή List εγγυάται τη διαθεσιμότητα αυτής της μεθόδου. Με άλλα λόγια σε σχέση με το παράδειγμά μας, τόσο η ArrayList όσο και η LinkedList διαθέτουν μέθοδο Iterator<E> iterator().

Στην main επομένως εκχωρούμε στην store την αναφορά του ArrayList που επέστρεψε η LoadData. Στη συνέχεια, όπως ακριβώς και στον κώδικα 14.1 λαμβάνουμε τον Iterator δια μέσου του οποίου προσπελαύνουμε τα στοιχεία του ArrayList. Όμως, τον κώδικα που προσπελαύνει τα στοιχεία της λίστας τον επαναλαμβάνουμε 2 φορές. Τη δεύτερη φορά έχουμε αλλάξει τη λίστα μας από ArrayList<Person> σε LinkedList<Person>. Ωστόσο ο κώδικας τρέχει σωστά και για τις δύο διαφορετικές δομές, καθώς και οι δύο είναι τύπου List και άρα και οι δύο παρέχουν Iterator. Να σημειωθεί ότι ο Iterator που παρέχεται σε κάθε περίπτωση βασίζεται σε μια κλάση που υλοποιεί τη διεπαφή Iterator. Η ίδια η κλάση όμως είναι διαφορετική για την ArrayList και διαφορετική για την LinkedList.

14.1.2 Comparator

Μερικές φορές έχουμε την ανάγκη να ταξινομήσουμε τα αντικείμενα μιας συλλογής κατά ποικίλους τρόπους. Για παράδειγμα, έστω ότι θέλουμε να εμφανίσουμε τα πρόσωπα της λίστας του κώδικα 14.2 καταρχάς ταξινομημένα κατά id και στη συνέχεια ταξινομημένα κατά επώνυμο και όνομα. Σε αυτές τις περιπτώσεις είναι χρήσιμη η διεπαφή Comparator<T>. Η διεπαφή δηλώνει μια αφηρημένη μέθοδο ως εξής:

```
int compare(T o1, T o2)
```

Συγκρίνει τις δύο παραμέτρους της. Επιστρέφει θετικό ακέραιο αν θεωρεί το o1 μεγαλύτερο από το o2, αρνητικό αν θεωρεί το o1 μικρότερο από το o2 και 0 αν τα θεωρεί ίσα.

Ας δούμε ένα παράδειγμα:

```
import gr.ihu.cs.lmous.OOJ.k12.v1.x.Person;
import java.util.Comparator;
import java.util.Iterator;
import java.util.List;

public class ComparatorDemo {

    private static class CompareById implements Comparator<Person> {

        @Override
        public int compare(Person o1, Person o2) {
            int id1 = Integer.decode(o1.getId());
            int id2 = Integer.decode(o2.getId());

            return id1 - id2;

        }
    }

    private static class CompareByName implements Comparator<Person> {

        @Override
        public int compare(Person o1, Person o2) {
            String sName1 = o1.getsName();
            String sName2 = o2.getsName();
            String fName1 = o1.getfName();
            String fName2 = o2.getfName();
            int rVal = sName1.compareTo(sName2);
            if (rVal == 0) {
                return fName1.compareTo(fName2);
```

```
                }
                return rVal;
            }
        }

        static void prt(Iterator<Person> it) {
            while (it.hasNext()) {
                System.out.println(it.next());
            }
        }

        public static void main(String[] args) {
            List<Person> store = PolymorphismDemo.loadData();
            store.sort(new CompareById());
            prt(store.iterator());
            System.out.println("---------------------");
            store.sort(new CompareByName());
            prt(store.iterator());
        }
    }
```

Κώδικας 14.3 *Χρήση Comparator για ταξινόμηση λίστας*

Στην main φορτώνουμε καταρχάς τα πρόσωπα του κώδικα 14.2 στην store. Στη συνέχεια καλούμε την sort. Η συνάρτηση είναι υπερφορτωμένη. Έχουμε ήδη χρησιμοποιήσει την έκδοση που δεν έχει παραμέτρους. Εδώ χρησιμοποιούμε την έκδοση που απαιτεί ένα Comparator ως παράμετρο. Αρχικά περνάμε ένα αντικείμενο της εσωτερικής κλάσης CompareById. Η CompareById υλοποιεί τη διεπαφή Comparator<Person>, δηλαδή τα αντικείμενά της είναι Comparators για αντικείμενα Person. Επομένως, η CompareById πρέπει να παρέχει υλοποίηση της μοναδικής αφηρημένης μεθόδου της διεπαφής, δηλαδή της compare. Έτσι η sort μπορεί με ασφάλεια να καλέσει από τον Comparator την compare ώστε να μπορέσει να ταξινομήσει τα αντικείμενα Person κατά id.

Στη συνέχεια, η prt λαμβάνει ως παράμετρο τον Iterator της store μέσω του οποίου προσπελαύνει και εμφανίζει ένα προς ένα τα αντικείμενα της store.

Αμέσως μετά ταξινομείται η store με τον CompareByName οπότε η επόμενη κλήση της prt εμφανίζει τα πρόσωπα ταξινομημένα κατά επώνυμο και όνομα. Προσέξτε την εσωτερική κλάση CompareByName. Καταρχάς συγκρίνει δύο πρόσωπα κατά επώνυμο. Αν βρεθούν τα επώνυμα ίδια, τότε συγκρίνει κατά μικρό όνομα.

14.2 Υλοποίηση διεπαφής

Σε αυτήν την ενότητα παρουσιάζουμε ένα παράδειγμα δήλωσης και υλοποίησης μιας διεπαφής. Πιο συγκεκριμένα δηλώνουμε μια διεπαφή που προσομοιώνει τον Iterator της Java αλλά είναι εξειδικευμένη ειδικά για συλλογές του τύπου String. Αυτή η επιλογή γίνεται για καθαρά εκπαιδευτικούς λόγους καθώς μια τέτοια διεπαφή ορθότερα θα ήταν παραμετροποιημένος τύπος. Παράλληλα με τη διεπαφή παρουσιάζεται και μια κλάση που αξιοποιεί τη διεπαφή. Ας ξεκινήσουμε με την παρουσίαση της διεπαφής:

```
public interface MyIterator {

    public boolean hasNext();
    public String next();
}
```

Κώδικας 14.4 *Η διεπαφή MyIterator*

Όπως φαίνεται στον κώδικα 14.4, μια διεπαφή ορίζεται παρόμοια με μία κλάση μόνο που χρησιμοποιείται η λέξη-κλειδί interface αντί της class. Παράλληλα οι μέθοδοι της διεπαφής είναι αφηρημένες.
Ο κώδικας που ακολουθεί παρουσιάζει την εφαρμογή της MyIterator:

```
public class ArrayListOfStrings {
```

```
private String[] store = new String[10];
private int index = 0;

private class ArrayListIterator implements MyIterator {

    private int current = 0;

    @Override
    public boolean hasNext() {
        return current < size();
    }

    @Override
    public String next() {
        return get(current++);
    }
}

private String[] increment(String[] store) {
    String[] rVal = new String[store.length + 10];
    System.arraycopy(store, 0, rVal, 0, store.length);
    return rVal;
}

public void add(String s) {
    if (index == store.length) {
        store = increment(store);
    }
    store[index++] = s;
}

public String remove(int idx) {
    if (idx > index || idx < 0) {
        return null;
    }
    String rVal = store[idx];
    for (int i = idx; i < index; i++) {
        store[i] = store[i + 1];
    }
    index--;
    return rVal;
}

public String removeLast() {
    return remove(index - 1);
}

public String get(int idx) {
    if (idx >= index || idx < 0) {
        throw new IndexOutOfBoundsException();
    }
    return store[idx];
}

public boolean isEmpty() {
    return index == 0;
}

public int size() {
    return index;
```

```
    }

    public MyIterator myIterator() {
        return new ArrayListIterator();
    }

    public static void main(String[] args) {
        ArrayListOfStrings list = new ArrayListOfStrings();
        list.add("A");
        list.add("B");
        list.add("C");
        list.add("D");
        MyIterator it = list.myIterator();
        while (it.hasNext()) {
            System.out.print(it.next());
        }
        System.out.println();
    }

}
```

Κώδικας 14.5 *Υλοποίηση MyIterator και ArrayListOfStrings*

Καταρχάς, ας μελετήσουμε την κλάση ArrayListOfStrings. Πρόκειται για μια απλή συλλογή. Ως αποθηκευτικό χώρο χρησιμοποιεί τον πίνακα store που αρχικά δημιουργείται με μήκος 10. Η index δείχνει στην πρώτη θέση του πίνακα που είναι ελεύθερη και μπορεί να αποθηκευτεί μία νέα συμβολοσειρά. Κατά τη δημιουργία αντικειμένου ArrayListOfStrings, η index είναι ίση με 0 που σημαίνει ότι ο πίνακας είναι άδειος. Έτσι η μέθοδος isEmpty επιστρέφει true μόνο εφόσον η index είναι ιση με 0. Η add ελέγχει αν υπάρχει χώρος στον store. Αν όχι καλεί την increment και αυξάνει τη χωρητικότητα του store κατά 10. Η remove διαγράφει το στοιχείο στη θέση idx από τον εσωτερικό πίνακα μετακινώντας όλα τα στοιχεία από τη θέση idx και μετά μία θέση πριν την τρέχουσα θέση τους. Αυτό βέβαια μπορεί να γίνει εφόσον κληθεί με παράμετρο μεγαλύτερη από το 0 και μικρότερη από την index. Ενημερώνει κατάλληλα την index και επιστρέφει το στοιχείο που διαγράφηκε. Η get επιστρέφει το στοιχείο στη θέση idx και η size τον αριθμό των συμβολοσειρών της ArrayListOfString.

Ας πάμε τώρα στην myIterator. Η τυπική τιμή επιστροφής της είναι MyIterator. Έτσι επιστρέφει ένα αντικείμενο της κλάσης ArrayListIterator που υλοποιεί τη διεπαφή MyIterator. Η ArrayListIterator διατηρεί τον δείκτη current που καθορίζει ποιο είναι για το στιγμιότυπο τύπου MyIterator το τρέχον στοιχείο της ArrayListOfString. Κατά τη δημιουργία των αντικειμένων ArrayListIterator, ο δείκτης current αρχικοποιείται στο 0. Έτσι κάθε νέο αντικείμενο ArrayListIterator θεωρεί ως πρώτο στοιχείο αυτό που βρίσκεται στη θέση 0 του store. Η hasNext επιστρέφει true εφόσον ο current είναι μικρότερος από την τιμή που επιστρέφει η size. Η δε next() επιστρέφει το στοιχείο στη θέση current του store και αυξάνει τον current κατά 1.

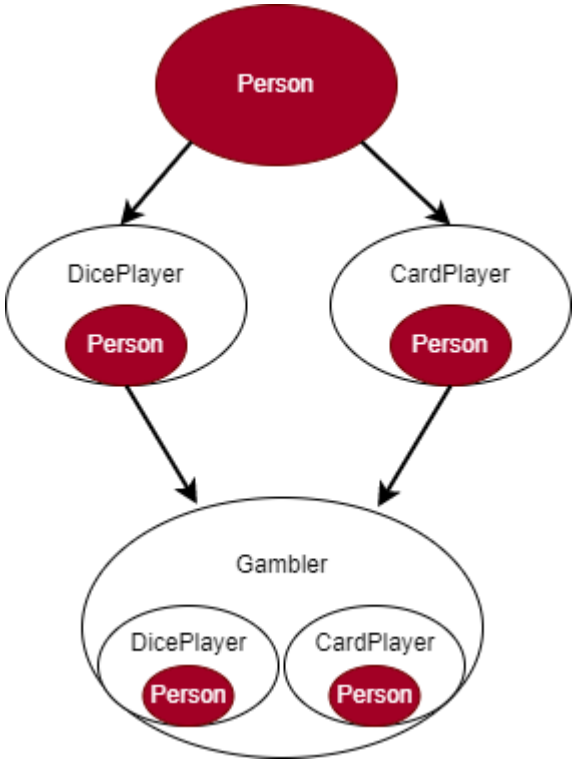
Η έξοδος της main είναι ABCD.

Ας σημειωθεί πως η ArrayListIterator θα μπορούσε να υλοποιεί την προκαθορισμένη διεπαφή Iterator της Java. Χρησιμοποιήθηκε εδώ η MyIterator ως μια ευκαιρία για να επιδειχθεί πως δηλώνουμε μια δική μας διεπαφή.

14.3 Πολλαπλή κληρονομικότητα

Ας υποθέσουμε πως έχουμε μια κλάση που αναπαριστά παίκτες ζαριών, έστω την DicePlayer, και μία κλάση που αναπαριστά χαρτοπαίκτες, έστω την CardPlayer. Έστω ότι χρειαζόμαστε και μία κλάση τα αντικείμενα της οποίας είναι ταυτοχρόνως και χαρτοπαίκτες και παίκτες ζαριού, έστω Gambler. Θα ήταν ίσως βολική η δυνατότητα η κλάση Gambler να κληρονομεί και την CardPlayer και την DicePlayer, δηλαδή η Gambler να έχει 2 γονικές κλάσεις. Εύκολα μπορούμε να σκεφτούμε παραδείγματα που ενδεχομένως θα ήταν χρήσιμο μια κλάση να έχει περισσότερες ακόμη και από 2 γονικές. Η σχέση κατά την οποία μία κλάση διαθέτει περισσότερες από μία γονικές είναι γνωστή ως πολλαπλή κληρονομικότητα (multiple inheritance).

Η πολλαπλή κληρονομικότητα μεταξύ κλάσεων χαρακτηρίζεται από το πρόβλημα του διαμαντιού (the diamond problem). Ανακαλέστε από την ενότητα 13.1 πως κάθε αντικείμενο παράγωγης κλάσης περιλαμβάνει ένα αντικείμενο της γονικής.



Σχήμα 14.1 *Το πρόβλημα του διαμαντιού*

Προσέξτε τώρα στο σχήμα 14.1 τι συμβαίνει αν η κλάση DicePlayer και η CardPlayer είναι παράγωγες της Person. Ένα αντικείμενο Person εσωκλείεται σε κάθε στιγμιότυπο της DicePlayer και της CardPlayer. Έτσι κάθε αντικείμενο της Gambler καταλήγει να περιλαμβάνει δύο αντικείμενα της Person. Αυτό βέβαια είναι μη αποδεκτό, καθώς αυξάνει ανώφελα τον όγκο των αντικειμένων Gambler και θέτει το ακόλουθο ερώτημα: Τι θα πρέπει να συμβεί αν αντικείμενο της Gambler καλέσει μια μέθοδο που αρχικά ορίζεται στην Person, π.χ. την getfName. Προσέξτε πως η getfName κληρονομείται και στην DicePlayer και στην CardPlayer. Επίσης λάβετε υπόψη πως η μέθοδος μπορεί να επανοριστεί ή όχι τόσο στην DicePlayer όσο και στην CardPlayer. Ποια getfName θα κληρονομήσει η Gambler; Επιπλέον, οι μεταβλητές στιγμιότυπου της Parent έχουν δημιουργηθεί και λάβει τιμές τόσο κατά τη δημιουργία του αντικειμένου DicePlayer όσο και κατά τη δημιουργία του CardPlayer. Ποιες μεταβλητές θα κληρονομήσει η Gambler; Αν υποθέσουμε πως η Gambler κληρονομεί τις μεταβλητές της Parent που δημιουργήθηκαν κατά την κατασκευή του αντικειμένου DicePlayer, οι μέθοδοι που κληρονομήθηκαν από την CardPlayer θα μπορούν να επενεργούν σε αυτές τις μεταβλητές;

Οι διάφορες σύγχρονες γλώσσες έχουν δώσει διαφορετική απάντηση στο πρόβλημα του διαμαντιού. Ας αποσαφηνίσουμε όμως κάποιες έννοιες σχετικά με την κληρονομικότητα ώστε να διευκολυνθούμε για να κατανοήσουμε τη λύση της Java.

Η κληρονομικότητα μπορεί να διακριθεί σε τρεις τύπους [2], την κληρονομικότητα κατάστασης (state inheritance), την κληρονομικότητα υλοποίησης (implementation inheritance) και την κληρονομικότητα τύπου (type inheritance). Η κληρονομικότητα κατάστασης αφορά τις μεταβλητές που κληρονομούνται από τη γονική, η κληρονομικότητα υλοποίησης τις υλοποιήσεις των μεθόδων που κληρονομούνται από τη γονική και τέλος η κληρονομικότητα τύπων αφορά τους τύπους που κληρονομούνται από τη γονική. Η κληρονομικότητα μεταξύ των κλάσεων περιλαμβάνει και τους τρεις τύπους κληρονομικότητας. Το προβλήματος του διαμαντιού όμως προκύπτει αποκλειστικά σε σχέση με την κληρονομικότητα κατάστασης και υλοποίησης. Επιπλέον, η κληρονομικότητα κατάστασης και υλοποίησης δεν είναι αναγκαίες προϋποθέσεις για την ανάπτυξη πολυμορφικού κώδικα. Αντίθετα η κληρονομικότητα τύπου είναι αναγκαία προϋπόθεση για την υποστήριξη πολυμορφικού κώδικα. Η Java δεν υποστηρίζει πολλαπλή κληρονομικότητα κατάστασης και υλοποίησης. Έτσι αντιμετωπίζει το πρόβλημα του διαμαντιού. Υποστηρίζει όμως πολλαπλή κληρονομικότητα τύπου δια μέσου των διεπαφών. Πράγματι οι διεπαφές διαθέτουν μόνο αφηρημένες μεθόδους και δεν επιτρέπουν τη δήλωση μεταβλητών στιγμιότυπου. Μία κλάση της Java μπορεί να κληρονομήσει μόνο από μία κλάση αλλά μπορεί να υλοποιήσει απεριόριστο αριθμό διεπαφών. Τα αντικείμενα της κλάσης συνδέονται με τη σχέση “είναι ένα” με κάθε διεπαφή που η κλάση υλοποιεί.

Ας δούμε τώρα πώς μπορούμε να αναπτύξουμε την κλάση Gambler στο πλαίσιο της Java έτσι ώστε κάθε αντικείμενο Gambler να είναι ταυτοχρόνως Person και DicePlayer και CardPlayer. Θα χρειαστούμε καταρχάς μια βοηθητική κλάση που αναπαριστά το ζάρι και δύο διεπαφές.

```
import gr.ihu.cs.lmous.OOJ.k11.v1.x.Lib;
import java.util.Arrays;

public class Dice {
    private final double[] distribution;

    public Dice(double[] distribution) {

        if (distribution.length != 6
                || !Lib.approximateEquals(Lib.sum(distribution), 1)) {
            throw new RuntimeException();
        }
        this.distribution = distribution;
    }

    public Dice() {
        distribution=new double[6];
        Arrays.fill(distribution, 1/6d);
    }

    public int roll() {
        double value = Lib.gen.nextDouble();
        double sum = 0;

        for (int i = 0; i < distribution.length; i++) {
            sum += distribution[i];
            if (value <= sum) {
                return i + 1;
            }
        }
        return 0;
    }
}

public interface DicePlayerI {

    public int roll(Dice dice);
}

import java.util.ArrayList;

public interface CardPlayerI {

    public Card pickTop(Deck deck);

    public ArrayList<Card> openCards();
}
```

Κώδικας 14.6 *Η κλάση Dice και οι διεπαφές DicePlayerI και CardPlayerI*

Σημειώστε πως η κλάση Dice και οι διεπαφές DicePlayerI και CardPlayerI πρέπει να τοποθετηθούν σε διαφορετικά αρχεία. Έχοντας στη διάθεσή μας τις δύο αυτές διεπαφές και την κλάση Dice, μπορούμε να ορίσουμε την κλάση Gambler που παρουσιάζεται στον κώδικα 14.7. Σημειώστε πως εδώ η κατανομή των πιθανοτήτων για το αποτέλεσμα του ζαριού δεν είναι πλέον χαρακτηριστικό του παίκτη αλλά του ζαριού που χρησιμοποιεί:

```
import java.time.LocalDate;
```